॥ श्रीगणेशाय नमः ॥

Arrays and Strings

268. Missing Number

Easy | Topics | Companies

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return *the only number in the range that is missing from the array.*

Example 1:

```
Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
```

Brute-force:
- for every # in that range, iterate and check.

$O(n^2)$

Better approach:
sort and linear search.

```
def missingNumber(nums, n):
    nums.sort()
    for i in range(0, n+1):
        if nums[i] != i:
            return i
```

$O(n \log n + n) = O(n \log n)$
space $= O(1)$

Better approach: hashing

All elements are in 0 to n. Take a bool vector of size n+1 (all values FALSE). Then iterate #s and mark present values true.

```
int missing_number(vi &nums) {
    int n = nums.size();
    vector<bool> hash(n+1, false);
    for (auto num : nums) {
        hash[num] = true;
    }
    FOR(i, 1, n+1)
        if (!hash[i])
            return i;
    return 0;
}
```

Time = O(n)
space = O(n)

Optimal approach:

Sum of #s $= \frac{n(n+1)}{2}$

{0, 1, 2, 3, 4} —Expected sum→ $\frac{4 \cdot 5}{2} = 10$

{0, 1, 2, 4} —actual sum→ 7

→ Diff = expected sum − actual sum = 10 − 7 = 3

Time = O(n)
space = O(1)

Best soln:

Sum and product are slightly costlier.

{3, 2, 0, 5, 1} n = 5

(3 ^ 2 ^ 0 ^ 5 ^ 1) ^ (0 ^ 1 ^ 2 ^ 3 ^ 4 ^ 5) ← XOR all #s in [0, n]

same elements will XOR to zero.
and zero ^ some_number = some_number.

The missing # has no pair.

XOR:

$a \oplus a = 0$

$a \oplus 0 = a$

```
  110110
⊕ 110110
--------
  000000

  110110
⊕ 000000
--------
  110110
```

X	Y	X⊕Y
T	T	F
T	F	T
F	T	T
F	F	F

```
int missing_number(vi &nums) {
    int ans = 0;
    for (auto &num : nums)
        ans ^= num;

    for (int i = 1; i <= nums.size(); i++)
        ans ^= i;

    return ans;
}
```

Still
time = O(n)
space = O(1)

But XOR is cheaper.

Subarrays and subsequences

Every sub-array is a sub-sequence, but not vice-versa.



sub-array [can be empty as well] [can be defined using start and end indices i & j]

2 3 1 7 ← sub-sequence [elements taken in order]

of sub-arrays:

[1, 2, 3, 4]

length 0 : 1
1 : 4
2 : 3
3 : 2
4 : 1
] 10] 11 (if we consider the empty one)

↳ selecting 2 indices i & j s.t. $0 \le i \le j \le 3$.

$i = 0$: $j = 0, 1, 2, 3$
1 : 1, 2, 3
2 : 2, 3
3 : 3

$^nC_2 + n$

$4 + 3 + 2 + 1 = 10$ $\quad \frac{n(n+1)}{2}$

of sub-sequences $= \frac{n(n+1)}{2} = O(n^2)$

of sub-sequences :

[1, 2, 3, 4]

→ 2^4 (if you count the empty one)

(| power set |

= 16

$^4C_0 + {}^4C_1 + {}^4C_2 + {}^4C_3 + {}^4C_4 = 2^4 = 16$

of subsequences = 2^n

Consider arr[n]. How many pair of elements i, j exist s.t. $i, j \in$ arr and $i \neq j$?

→ $^nC_2$ pairs in total. $O(n^2)$.

of triplets : $^nC_3$: $O(n^3)$

560. Subarray Sum Equals K

Medium | Topics | Companies | Hint

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to* `k`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

```
Input: nums = [1,1,1], k = 2
Output: 2
```

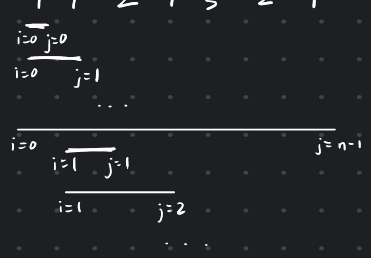
Example : [1, 1, 2, 1, 3, 2, 1, 1, 1, 4, 2] k = 6

[2, 1, 3]
[1, 3, 2]
[2, 1, 1, 1]
[1, 1, 4]
[4, 2]

① Brute-force : check every sub-array.

```
ans = 0
for (i : 0 → n-1) :
    for (j : i → n-1) :
        sum = 0
        for (l : i → j) :
            sum += arr[l]
        if sum == k : ans += 1
return ans
```

e.g. 1 1 2 1 3 2 1



$T(n) = O(n^3)$
space = $O(1)$

② 1 1 2 1 3 2 1

When we have calculated this sum already, we can calculate this sum by adding 1 element. Extra constant time. So why recalculate ?

```
// O(n²) solution
int subarraySum(vector<int>& nums, int k) {
    int n = nums.size();
    int count = 0;

    FOR(i, 0, n) {          ← for any i
        int sum = 0;
        FOR(j, i, n) {
            sum += nums[j];
            if (sum == k)
                count++;
        }
    }
    return count;
}
```

← Keep adding all the subsequent elements arr[j] one-by-one.

③ Prefix-sum :

```
idx: 0  1  2  3  4  5  6  7  8  9  10 11
     [1, 1, 2, 1, 3, 2, 1, 1, 1, 1, 4, 2]   k = 6
```

i = 3, j = 6 sum[i,j] = 7
i = 0, j = 10 sum[i,j] = 18
] ← expensive to calc. individually. O(n) for each such calculation

$Prefix[i] = \sum_{j=0}^{i} arr[j]$ ← like cumulative sum.

```
idx: 0  1  2  3  4  5  6  7  8  9  10 11
     [1, 1, 2, 1, 3, 2, 1, 1, 1, 1, 4, 2]
```

Prefix-sum array:
```
     [1  2  4  5  8  10 11 12 13 14 18 20]
```

sum[3,8] = Prefix[8] − Prefix[2]
= 13 − 4 = 9

or suffix[3] − suffix[9]

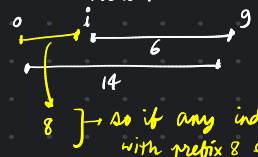
```
[0 1 2 3 ... 8]
-[0 1 2]
[3 ... 8]
```

soln :

```
idx: 0  1  2  3  4  5  6  7  8  9  10 11
     [1, 1, 2, 1, 3, 2, 1, 1, 1, 1, 4, 2]
```

Prefix-sum array:
```
idx: 0  1  2  3  4  5  6  7  8  9  10 11
     [1  2  4  5  8  10 11 12 13 14 18 20]
```

sum till 8 = 14
we'll check if any subarray ending here has sum 6.



→ so if any index i with prefix 8 exists, we get sum[i...9] = 6

we can use a hash_map to check how many times a prefix value has appeared till now so that we wouldn't need to traverse.

At every index :
ans = ans + map[sum − k]

Flaw : 1st index has 1 . [1, ...]
for this index we should check if hash map has 0 if we want sum 1.
So just keep a '0' in the hash-map.

```cpp
int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> mp;   mp[0] = 1;
    int sum = 0, res = 0;

    for (auto n: nums) {
        sum += n;
        res += mp[sum - k];
        mp[sum]++;
    }

    return res;
}
```

time : O(n)
space : O(n)

O(nlogn) ← map worst case, but happens rarely

1. Two Sum

Easy | Topics | Companies | Hint

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to* `target`.

You may assume that each input would have ***exactly* one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

```python
def twoSum(self, nums: List[int], target: int) -> List[int]:
    d = dict()
    for i,n in enumerate(nums):
        if target-n in d:
            return [d[target-n], i]
        d[n] = i
    return []
```

time = space = O(n).

2 pointer approach:

a = [-1, 3, -2, -6, 10, 5, -3] k = 3
0 1 2 3 4 5 6

→ (-2) + (5) = 3. Reqd. indexes are [2, 5]

sorted a = [-6, -3, -2, -1, 3, 5, 10]
i j

Since array is sorted:

right now, -6 + 10 = 4

i stores smallest value and goes to largest.

j goes largest → smallest.

curr_sum = 4, we want 3.

If we move i →, sum will increase.

j ←, sum will decrease.

so let's shift j.

[-6, -3, -2, -1, 3, 5, 10]
i j

now sum = -1

now we want a larger sum.

move i →

[-6, -3, -2, -1, 3, 5, 10]
i j

now Sum = 2. < 3.

move i →

[-6, -3, -2, -1, 3, 5, 10]
i j

Sum = 2

reqd. sum.

(-2, 5) are the reqd. elements.

```python
def twoSum(self, nums: List[int], target: int) -> List[int]:
    i = 0
    j = len(nums) - 1
    for k in range(len(nums)):
        nums[k] = (nums[k], k)
    nums.sort()

    while i < j:
        if nums[i][0] + nums[j][0] > target:
            j -= 1
        elif nums[i][0] + nums[j][0] < target:
            i += 1
        else:
            return [nums[i][1], nums[j][1]]
```

T = O(n log n + n) = O(n log n)

```cpp
vector<int> twoSum(vector<int>& nums, int target) {
    vector<pair<int, int>> vals;
    for (int i=0; i < nums.size(); i++)
        vals.push_back({nums[i], i});
    sort(vals.begin(), vals.end());

    int i=0, j=vals.size()-1;
    while (i < j) {
        if (vals[i].first + vals[j].first < target) i += 1;
        else if (vals[i].first + vals[j].first > target) j -= 1;
        else return vector<int> {vals[i].second, vals[j].second};
    }
    return vector<int>{};
}
```

3 sum:

Fix i. For every i, find j and k using 2 pointer approach.

$O(n \log n + n^2)$ Ily $O(n \log n + n^3)$ for 4 sum.

Dutch national flag algorithm

Problem: sort an array containing 3 distinct values that are repeated.

AABACABCC → AAAAABBCC

0012011201 → 00001111 22

① Approach:

- count # of 0s, 1s, and 2s. time = O(n) space = O(1).
- Rewrite array.

② use 3 pointers:

[0000001111111222222]

low mid mid high

2, 1, 0, 0, 1, 1, 0, 2, 0, 1, 2, 1

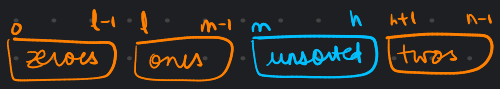
l m h

0 → l-1 : all 0s

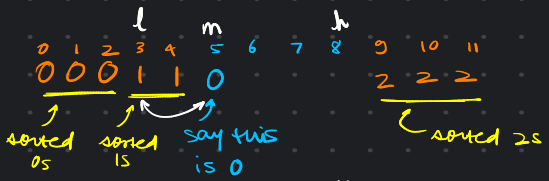
l → m-1 : all 1's

m → h : unsorted array]→ m iterates and then places elements in either of the 3 sections.

h+1 → n-1 : all 2's



e.g.



swap 2 elements



unsorted arr : m to h

So we do all of this as long as m ≤ h.

Initial values:

l=0, m=0, h= arr.size() - 1

Time = O(n), space = O(1).

so If a[m]==0:

swap(a[l], a[m])

l+=1; m+=1



If a[m] is 1,
Just move m since 1 is already sorted.

if a[m]==1:

m+=1



If 2,
Swap these 2
We don't know what a[h] is,
so let's not increase m.
m will be dealt with in the next iteration.

if a[m]==2: swap a[m], a[h] ; h -= 1;

```
def sortColors(self, nums: List[int]) -> None:
    """
    Do not return anything, modify nums in-place instead.
    """
    l = m = 0
    h = len(nums) - 1

    while m <= h:
        if nums[m] == 0:
            nums[l], nums[m] = nums[m], nums[l]
            l += 1
            m += 1
        elif nums[m] == 1:
            m += 1
        elif nums[m] == 2:
            nums[m], nums[h] = nums[h], nums[m]
            h -= 1
```

NOTE: Both these methods are unstable sorts.

Moore's voting algorithm

Problem: Find the majority element in an array.

$> \frac{n}{2}$ times

(not even $\geq \frac{n}{2}$ times)

e.g. $[3,3,3,2,2,1]$ $n=6$, $n/2=3$

→ no majority

$[3,2,1,3,2,1,3,3]$ $n=8$, $n/2=4$

→ no majority

$[3,3,1,3,2,1,3,3]$ $n/2=4$

3 occurs 5 times

→ 3 is majority.

① Brute-force:

```
for (i: arr) {
    c = 0
    for (j : arr)
        if (i == j) c += 1
    if c > n/2 return i;
}
```

$T(n) = O(n^2)$

Space $= O(1)$.

② Freq. count.

```
map<int, int> freq;
for (i: arr)
    freq[i] += 1;
for (i: freq)
    if freq[i] > n/2 :
        return i
```

assuming unordered map takes O(1).

time $= O(n)$

space $= O(n)$.

③ Optimal soln:

$[3,3,1,3,2,1,3]$

If all the other elements try to cancel out the majority element, it'll still be left out.

↳ Doesn't happen for highest freq, only majority elements.

$[1,3,2,3,3,3,2,3]$ count = 0, majority = -1

$[1,3,2,3,3,3,2,3]$ count = 1, majority = 1

$[\not1,\not3,2,3,3,3,2,3]$ count = 0, majority = ✗

$[\not1,\not3,\not2,3,3,3,2,3]$ count = 1, majority = ✗

$[\not1,\not3,\not2,\not3,3,3,2,3]$ count = 0, majority = ✗ ≠

$[\not1,\not3,\not2,\not3,3,3,2,3]$ count = 1, majority = ✗ ≠ 3

$[\not1,\not3,\not2,\not3,\not3,3,2,3]$ count = 0, majority = ✗ ≠ 3

$[\not1,\not3,\not2,\not3,\not3,\not3,2,3]$ count = 1, majority = ✗ ≠ 3

$[\not1,\not3,\not2,\not3,\not3,\not3,\not2,3]$ count = 2, majority = ✗ ≠ 3

↳ if a majority exists, it'll be this

↳ use 1 pass to check if this is majority.

$O(n+n) = O(n)$

169. Majority Element

Easy | Topics | Companies

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

```python
def majorityElement(self, nums: List[int]) -> int:
    majority = None
    count_majority = 0

    for i in nums:
        if count_majority == 0:
            majority = i
            count_majority = 1
        elif i == majority:
            count_majority += 1
        else:
            count_majority -= 1

    return majority
```

t = O(n)
s = O(1)

you could add a check:

```python
count = 0
for i in nums:
    if i == majority:
        count += 1

if count > len(nums)//2:
    return majority
else:
    return None
```

Maximum subarray sum

Kadane's algorithm

Problem: Find the sub-array with the maximum sum.

[-2 1 -3 4 -1 2 1 -5 4]

↳ sum = 6

Like in subarray sum k problem:

① Brute-force: check every sub-array.

```
max_sum = 0
for(i: 0 → n-1):
    for(j: i → n-1):
        sum = 0
        for(l: i → j):
            sum += arr[l]
        if sum > max_sum:
            max_sum = sum
return max_sum
```

e.g. 1 1 2 1 3 2 1

i=0 j=0
i=0 j=1
...
i=0 j=n-1
i=1 j=1
i=1 j=2
...

$T(n) = O(n^3)$
Space : $O(1)$

② 1 1 2 1 3 2 1

↳ When we have calculated this sum already, we can calculate this sum by adding 1 element. Extra constant time. So why recalculate?

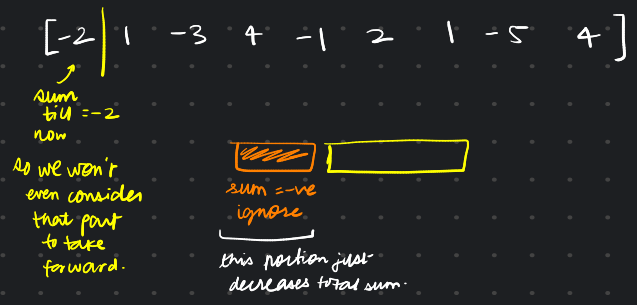
```
// O(n²) solution
int subarraySum(vector<int>& nums, int k) {
    int n = nums.size();
    int count = 0;
    max_sum = -∞
    FOR(i, 0, n) {                    ← for any i
        int sum = 0;
        FOR(j, i, n) {
            sum += nums[j];           ← keep adding all the subsequent elements arr[j] one-by-one.
            if (sum == k)
                count++;
            max_sum = max(max_sum, sum)
        }
    }
    return count;
}
```

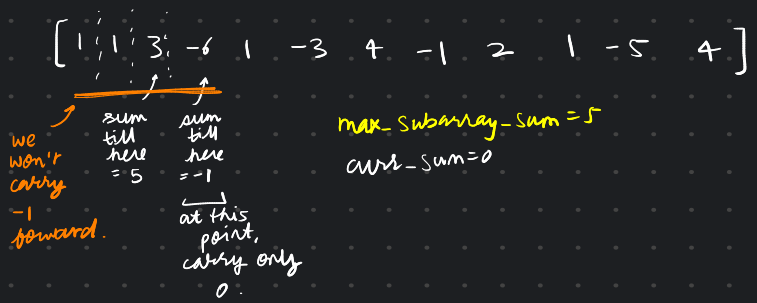
```
int maxSubArray(vector<int>& nums) {
    int sum = 0, max_sum = -INT_MAX;
    int n = nums.size();

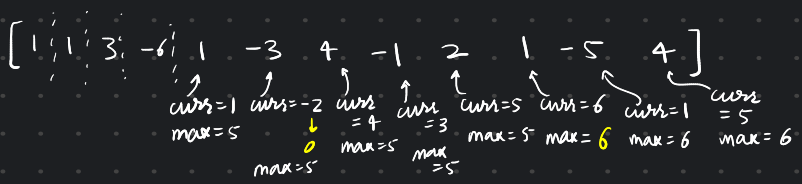
    FOR(i, 0, n) {
        sum = 0;
        FOR(j, i, n) {
            sum += nums[j];
            max_sum = max(sum, max_sum);
        }
    }
    return max_sum;
}
```

← $O(n^2)$ soln. Space = $O(1)$.

Kadane's algorithm








```cpp
int maxSubArray(vector<int>& nums) {
    int curr_sum = 0, max_sum = INT_MIN;

    for (auto n: nums) {
        curr_sum += n;
        max_sum = max(max_sum, curr_sum);
        if (curr_sum < 0)
            curr_sum = 0;
    }

    return max_sum;
}
```

time = O(n)
space = O(1).

Sort in specific order

Difficulty: **Basic** Accuracy: **52.55%** Submissions: **39K+** Points: **1**

Given an array **A** of positive integers. Your task is to sort them in such a way that the first part of the array contains odd numbers sorted in descending order, rest portion contains even numbers sorted in ascending order.

Example 1:

Input:
N = 7
Arr = {1, 2, 3, 5, 4, 7, 10}
Output:
7 5 3 1 2 4 10

Odds evens

Approach 1 : split into 2 lists. sort separately.

Approach 2: modify merge sort.

```
vector<int> mergeSort(vector<int> &nums, int l, int r) {
    int mid = (l+r) /2;

    vector<int> left = mergeSort(nums, l, mid);
    vector<int> right = mergeSort(nums, mid+1, r);

    return merge(left, right);
}
```

merge func^n:

say left = [3 1 2] right = [7 4 6]

i j

Iterate through all odds:

res.push_back(greater)

and

greater index += 1;

```
while (left[i] % 2 != 0 && right[j] % 2 != 0) {
    if (left[i] > right[j]) {
        res.push_back(left[i]);
        i += 1;
    }
    else {
        res.push_back(right[j]);
        j += 1;
    }
}
```

after this only 1 of the 2 arrays has any odd #s left.

```
while (left[i] % 2 != 0) {
    res.push_back(left[i]);
    i += 1;
}
while (right[j] % 2 != 0) {
    res.push_back(right[j]);
    j += 1;
}
```

push remaining odds.

If both odd:
insert greater
If even & odd:
insert odd
If both even:
insert smaller.
Then when one list is exhausted,
insert all remaining.

```cpp
vector<int> merge(vector<int> left, vector<int> right) {
    int l=0, r=0;
    vector<int> res;

    while (l < left.size() && r < right.size()) {
        // one even, one odd
        if (left[l]%2==0 and right[r]%2==1) res.push_back(right[r++]);
        else if (left[l]%2==1 and right[r]%2==0) res.push_back(left[l++]);

        // both odd
        else if (left[l]%2==1)
            res.push_back(
                    (left[l] > right[r]) ? left[l++] : right[r++]
                );

        // both even
        else if (left[l]%2==0)
            res.push_back(
                    (left[l] < right[r]) ? left[l++] : right[r++]
                );
    }

    // remaining elements
    while (l < left.size()) res.push_back(left[l++]);
    while (r < right.size()) res.push_back(right[r++]);

    return res;
}
```

Problem: Given a text T and pattern P. Check if P exists in T. If yes, print all the occurrences of P in T.

Sliding window

T = abacabab aac baba
0 4 6 12 ← aba is a substring.

P = aba

sliding window essentially matches all size 3 substrings with P. Brute-force.

|T| = n |P| = m len(P)

of such substrings = n-m+1.

time to compare once = O(m).

T = O((n-m+1)(m)) ≈ O(nm) (as n >> m)

```python
def strStr(self, haystack: str, needle: str) -> int:
    m = len(needle)
    n = len(haystack)

    if m == 0:
        return 0

    if m > n:
        return -1

    for i in range(n-m+1):
        j = 0
        while j < m:
            if needle[j] != haystack[i+j]:
                break
            j += 1
        if j == m:
            return i

    return -1
```

796. Rotate String

Easy | Topics | Companies

Given two strings `s` and `goal`, return `true` *if and only if* `s` *can become* `goal` *after some number of* ***shifts*** *on* `s`.

A **shift** on `s` consists of moving the leftmost character of `s` to the rightmost position.

- For example, if `s = "abcde"`, then it will be `"bcdea"` after one shift.

Example 1:

```
Input: s = "abcde", goal = "cdeab"
Output: true
```

Example 2:

```
Input: s = "abcde", goal = "abced"
Output: false
```

```python
def rotateString(self, s: str, goal: str) -> bool:
    if len(s) != len(goal):
        return False

    n = len(s)

    for i in range(n):
        flag = True
        for j in range(n):
            if s[(i+j)%n] != goal[j]:
                flag = False
                break
        if flag:
            return True

    return False
```

OR if we use the function for substrings:

s = abcde

s' = abcdeabcde

g = cdeab

strStr(s', g)

```python
class Solution:
    def rotateString(self, s: str, g: str) -> bool:
        if len(s)!=len(g):
            return False
        return goal in s+s
```

Hashing of strings

We'll map multiple strings to the same integer with a low probability of collision.
That would make comparison betn strings a lot easier (just compare hash values).

Polynomial hashing

$$\text{hash}(S) = S[0] + S[1]p + S[2]p^2 + S[3]p^3 + \dots$$
$$= \left(\sum_{i=0}^{n-1} S[i]p^i\right) \bmod m$$

S['a'] = 1
S['b'] = 2
⋮

p = 3:

h("a") = $1 \cdot 3^0 = 1$
h("b") = 2
h("z") = 3
h("aa") = $S['a'] \cdot 3^0 + S['a'] \cdot 3 = 1 + 1 \cdot 3 = 4$

h('d') = 4 → collision

p = 31:

h('aa') = $1 \cdot 31^0 + 1 \cdot 31^1 = 32$
h('zz') = $26 + 26 \cdot 31 = 832$
h('aaa') = $1 \cdot 31^0 + 1 \cdot 31^1 + 1 \cdot 31^2 = 993$.

collisions will occur.
But they won't mostly affect us.

```
const int MOD = 1e9 + 7;
const int P = 31;
const int N = 1e5;
vll PP(N+1);
```

```
FOR(i, 1, N+1)
    PP[i] = (PP[i-1] * P) % MOD;
```

```
ll poly_hash(string &s) {
    ll res = 0;
    FOR(i, 0, s.length())
        res = (res + (s[i] - 'a' + 1) * PP[i]) % MOD;
    return res;
}
```

Problem: search duplicate strings in an array of strings.

len n, len ≤ m

Brute-force will take $O(n^2 m)$. sorting takes $O(nm \log n + nm) = O(nm \log n)$
(sorting; comparing only consecutive elements after sorting)

We can avoid the O(m) for comparison.

compute hash of all strings. For a size m string: ① pre-compute powers of p to m O(m)
② Hash value of 1 string O(m)
∴ Hashing n strings: O(mn)

Sort hash values and compare adj. values $O(n \log n)$

$$O(nm + n \log n)$$
$$\approx O(nm)$$